Jorg Brown <jorg.brown@gmail.com>
2019-01-21
Document P1228 $Id: proposal.html,v 1.50 2018/01/21 07:26:32 jorg Exp $

A proposal to add an efficient string concatenation routine to the Standard Library (Revision 1)

Revision history

- 2019-01-21: Publication in the Pre-Kona mailing as P1228.

I. Motivation

> *Why is this important? What kinds of problems does it address, and what kinds of programmers, is it intended to support? Is it based on existing practice?*

In 2004, I became aware that std::string's operator+ was hideously inefficient; adding 4 strings together involved the allocation (and destruction) of three temporary strings, and each time the entire contents of the string thus far was copied. In an attempt to make this faster, our compiler had inlined the memcpy code four times, resulting in more than 1,000 bytes of code to construct a string that was generally 30 or 40 bytes long. So I wrote a small string copying utility, StrCat, with the primary goal of reducing code size.

Initially, all the parameters to StrCat were char*/length pairs, similar to std::string_view. But along the way I discovered I could also inexpensively format integers with the same API. By using fixed-size buffers that could be put on the stack, rather than separate heap-allocated std::string temporaries, significant speed increases were obtained. Combined with the original technique of computing the length of the string in advance, and thus eliminating all unnecessary copies, StrCat's performance became second only to hand-coded character-by-character copy loops.

C++11's rvalue references have dramatically improved the efficiency of operator+, however the utility of a purpose-built concatenation utility has proven itself; there are more than 1.6 million calls to StrCat in Google's internal code base, and it was one of the most highly-requested APIs for the open-source "Abseil" project.

Also, with C++11, it became possible to streamline StrCat itself; variadic template functions removed the need for 26 overloads to support up to 26 parameters. std::initializer_list allows us to reduce its stack usage, as well as add user-extensibility. And C++17's to_char family of conversion routines handles the difficult task of efficiently converting numbers to characters.

Target Audience

Any programmer who has ever used std::string's "operator+" to concatenate strings, or formatted them with a printf-like API, is a potential user.

A. Uphill through the snow, both ways!

Traditional concatenation looks like this:

```
std::string full_body = prologue + main_str + epilogue;
```

Which creates a temporary string that contains `prologue + main_str`, then appends epilogue to that temporary string, before finally moving the temporary into full_body. Since the temporary is just big enough to hold `prologue + main_str`, appending epilogue generally results in a second allocation, and a copy of the existing data into the new allocation.

Very commonly, integers are part of the result string:

```
std::string err = "Write of " + std::to_string(num_bytes) + " bytes to " + filename + " failed";
```

A temporary string must first be created to store the result of to_string; then another is created to hold the combination of "Write of" and the temporary. Then three more appends occur before the second temporary is moved into err, and the first temporary is destroyed. Also, std::to_string uses printf, which then has to parse '%d' and consider locale information before proceeding.

As if that weren't bad enough, certain mistakes are perfectly legal C++:

```
std::string err = "Write of " + num_bytes + string(" bytes to ") + filename + " failed";
```

Here, the expression `"Write of " + num_bytes` merely increments the character pointer that once pointed to 'W'.

B. Now with a purpose-built concatenation routine.

With std::concat, the source code looks very similar, but is much more efficient:

```
  std::string err = std::concat("Write of ", num_bytes, " bytes to ", filename, " failed");
```

The core API of concat is quite simple:

```
template <typename... T>
string concat(const T&... t) {
  using internal::to_concat;
  return internal::concat_views({to_concat(t)...});
}
```

This time, a small stack buffer is set up, and num_bytes is stringified into it, using a routine that, unlike sprintf, doesn't need to first look up the current locale. Then the lengths of all 5 parameters are added together, and a string is allocated of exactly that size. Then the character data of each argument is copied into that string. On a 2.6GHz Intel Broadwell, the following timings were observed:

```
  285ns: std::string err = "Write of " + std::to_string(num_bytes) + " bytes to " + filename + " failed";
  175ns: snprintf(stack_buf, sizeof(stack_buf), "Write of %u bytes to %s failed", num_bytes, filename.c_str());
         std::string err = stack_buf;
  135ns: snprintf(stack_buf, sizeof(stack_buf), "Write of %u bytes to %s failed", num_bytes, filename.c_str());
   78ns: std::string err = std::concat("Write of ", num_bytes, " bytes to ", filename, " failed");
```

II. Impact On the Standard

> *What does it depend on, and what depends on it? Is it a pure extension, or does it require changes to standard components? Does it require core language changes?*

This proposal is a pure library extension. It does not require changes to any standard classes or existing functions.

It makes use of the `std::to_chars` functionality added in C++2017, to achieve good performance when writing to character buffers.

III. Design Decisions

> *Why did you choose the specific design that you did? What alternatives did you consider, and what are the tradeoffs? What are the consequences of your choice, for users and implementors? What decisions are left up to implementors? If there are any similar libraries in use, how do their design decisions compare to yours?*

A. Formatting of built-in integer types

Thankfully, almost everyone agrees how to format the integer types that are built-in to the language, other than the single-character types: The sign comes first, either "-" or "", and then the number itself, in base 10.

"char", however, is disputed. `operator<<(ostream&, char)` treats it as an actual character, while std::to_string(char) treats it the same as it would treat the same value, promoted to "int" type. Many code bases define custom integer types such as int8 or int_0_255 which are defined in terms of char, signed char, or unsigned char, but since these are mere typedefs, it is not possible to know what was truly meant. In my experience, it causes more mayhem when an integer is incorrectly treated as a character, than the other way around, therefore this paper treats "char", "signed char", and "unsigned char" as integers. For those rare cases where a character is truly desired, a simple "string(1, ch)" call will force the value to be treated as an actual character.

"bool" is treated as an integer as well, by both std::stream and std::to_string, therefore std::concat treats it that way as well, even though many would prefer the text "true" and "false". Making this decision slightly easier is the fact that users don't agree whether "true", "True", "TRUE", or "T" is the best stringified boolean representation for a "1" bit.

B. Formatting of built-in floating-point types

`stream::operator<<(float)` and `to_string(float)` return dramatically different results. In printf terminology, the former uses "%g" while the latter uses "%f". Consider the following code:

```
  cout << "positive normal float holds values from "
       << FLT_MIN << " to " << FLT_MAX << "\n";
```

```
  cout << "positive normal float holds values from "
       << std::to_string(FLT_MIN) << " to " << std::to_string(FLT_MAX) << "\n";
```

It produces this output:

```
positive normal float holds values from 1.17549e-38 to 3.40282e+38
positive normal float holds values from 0.000000 to 340282346638528859811704183484516925440.000000
```

The treatment of small values as though they were zero is unfortunate, but the bigger issue is that, for speed, concat uses a fixed-size buffer. And while all 32-bit IEEE-754 floats are represented in 12 characters or less with %g, %f uses up to 47 characters. For 64-bit doubles, it's even worse: %g is limited to 13 characters while %f produces up to 317 characters. Therefore concat opts for the stream behavior.

C. Support for user-defined types.

Using SFINAE to determine if a user-defined type has a "ToString" method was briefly considered, as was support for a type's existing `ostream operator<<()`. Both were rejected on the grounds that std::concat exists for performance, so the use of an API which is not performant should be apparent. Also, the use of a member function would not allow support for gcc / llvm's built-in __int128 type. Therefore we have chosen to call to_concat with each of the parameters to std::concat, and to cast each result to a std::string_view. This allows types with an upper bound for string representation size to return a fixed-size buffer, while types with a possibly-large representation can return a string.

D. Support for string types other than std::string.

As of this writing, the C++ strings library supports not just std::string, but also wstring, u16string, u32string, u8string, and pmr variants of each. Ideally std::concat would support all of them, but unfortunately, automatically determining what the result type of std::concat should be is not possible, since it's not required for any of the parameters to be indicative of string type. Therefore, to support other string types would require a further-templated concat function:

```
template <class CharT = char,
    class Traits = std::char_traits<CharT>,
    class Allocator = std::allocator<CharT>, typename... T>
basic_string<CharT, Traits, Allocator> basic_concat(const T&... t);
```

It would also require each user-defined to_concat function to be templated, as well as dealing with the fact that std::to_chars currently does not support anything other than "char".

Another solution would be to provide wconcat, u16concat, etc... which would also require custom user-defined to_concat functions. In the end, it seemed better to simply consider std::concat to be a low-level utility, and to consider that broader unicode support would likely call for locale support as well. Therefore, this paper currently opts out of support for string types other than std::string.

E. More access to the underlying code?

This proposal does not make the internal API for stringifying integers (e.g. `std::internal::to_concat(int)`) public, on the grounds that users can already use std::to_chars. It's conceivable that it would it prove useful in other situations, however.

S. Sketch of a sample implementation

```
namespace std {
namespace internal {

template <size_t max_size>
struct concat_buffer {
  std::array<char, max_size> data;
  unsigned short size;
  operator std::string_view() const { return {&data[0], size}; }
};

// The core: sums the lengths of the given views, creates a string of that size,
// and copies the views in.
string concat_views(std::initializer_list<std::string_view> views);

concat_buffer<16> to_concat(int i);
concat_buffer<16> to_concat(unsigned int i);
concat_buffer<32> to_concat(long i);
concat_buffer<32> to_concat(unsigned long i);
concat_buffer<32> to_concat(long long i);
concat_buffer<32> to_concat(unsigned long long i);
```

```
concat_buffer<16> to_concat(float);
concat_buffer<32> to_concat(double);

// Normal enums are already handled by the integer formatters.
// This overload matches only scoped enums.
template <typename T,
          typename = typename std::enable_if<
              std::is_enum<T>{} && !std::is_convertible<T, int>{}>::type>
auto to_concat(T e) {
  return to_concat(static_cast<typename std::underlying_type<T>::type>(e));
}

inline std::string_view to_concat(const std::string_view& sv) { return sv; }

}  // internal

template <typename... T>
inline __attribute__((always_inline)) string concat(const T&... t) {
  using internal::to_concat;
  return internal::concat_views({std::string_view{to_concat(t)}...});
}

}  // namespace std
```

IV. Proposed Additional Text

Header `<string>` synopsis

```
namespace std {
// [string.conversions], numeric conversions
template <typename... T> string concat(const T&... t);
template <typename... T> string concat(string&& dest, const T&... t);
} // namespace std
```

Numeric conversions

```
template <typename... T> string concat(const T&... t);
template <typename... T> string concat(string&& dest, const T&... t);
```

The variadic template function `concat` converts its parameters into character strings, and then returns a `string` object holding the concatenation of those character strings. The character strings hold the character representation of the values of their arguments: If an argument is of integral type, the conversion is performed as if by std::to_chars, with a base of 10. An enum parameter is first cast to its underlying integral type, and then converted. If an argument is of floating-point type, it is converted as if by std::to_chars, with chars_format::general and precision of 6. Arguments that can be implicitly converted to std::string_view will be converted to std::string_view, and then concatenated. If the expression "to_concat(arg)" is valid (that is, if ADL finds a to_concat overload in arg's namespace), the argument will first be passed to to_concat(), before being explicitly cast to std::string_view, and then concatenated.

The second form of concat performs the same operation as the first, but is often more efficient because it simply appends the arguments to dest, before returning dest. Specifically, if dest is already large enough, no memory allocation is needed. Like string::append, if an exception is thrown during reallocation, the second form will not change the "dest" string parameter at all.

V. Acknowledgements

- Thanks to Samuel Benza, who first introduced me to the concept of constructing an initializer_list from the result of a function call applied to each argument of a variadic function. And also, for first suggesting accepting an rvalue string as the first parameter, purely as an optimization.

VI. References

- Jens Maurer, "Elementary string conversions" (P0067R5)
- Abseil open-source C++ Library, "StrCat and StrAppend for String Concatenation"

VII. Random quotes

> Your average "float" is just random noise after the first six significant digits.

Jorg Brown, 2018.

Anyone who considers arithmetical methods of producing random digits is, of course, in a state of sin.

John von Neumann, 1951